

Methane & CO2 Tracker

Relatório da Fase 2 — Ingestão MQTT → Protobuf e Regra de Alerta

Escopo desta fase: serviço que assina o(s) tópico(s) MQTT onde o gateway 4G/LoRaWAN publica leituras dos sensores TDLAS, decodifica o payload binário Protobuf, valida e grava no TimescaleDB, e avalia uma regra de alerta por concentração sustentada acima do threshold.

Decisões de arquitetura desta fase

- Schema expandido de CH4-only para multi-gás: **readings.ppm_ch4** renomeado para **concentration_ppm**, com nova coluna **gas_type** (CH4/CO2) em readings e alerts. Migração 0002, não altera a 0001 original.
- Contrato Protobuf (*proto/sensor_reading.proto*) ganhou enum `GasType` e o campo `gas_type` — é a fonte única de verdade compartilhada com o firmware, atualizado em conjunto com docs/protocol-spec.md.
- Regra de alerta implementada como **sustentada por janela** (default 120s / 500ppm CH4 / 5000ppm CO2, configurável via .env), não como cruzamento instantâneo — consistente com a especificação original do produto. Sem estado em memória: reavalia a janela a partir do banco a cada leitura, então o serviço pode reiniciar sem perder o estado do alerta.
- Consumer MQTT (paho-mqtt) isola cada mensagem em try/except: payload malformatado é logado e descartado, nunca derruba o processo. Reconexão ao broker via `reconnect_delay_set` (backoff automático).

Arquivos entregues

Arquivo	Papel
proto/sensor_reading.proto	Contrato binário sensor -> cloud (atualizado)
backend/alembic/versions/0002_multi_gas_schema.py	Migração multi-gás
backend/app/models/orm.py	ORM atualizado (gas_type, concentration_ppm)
backend/app/ingestion/generated/sensor_reading_pb2.py	Código gerado do .proto
backend/app/ingestion/decoder.py	Bytes Protobuf -> dict (com lookup de sensor)
backend/app/ingestion/schemas.py	Validação Pydantic da leitura decodificada
backend/app/ingestion/mqtt_consumer.py	Serviço de ingestão MQTT
backend/app/alerts/rules.py	Regra de alerta sustentado por janela
mosquitto/config/mosquitto.conf	Config dev-only do broker (allow_anonymous)
backend/tests/test_db_connection.py	Teste standalone — executado, OK
backend/tests/test_proto_roundtrip.py	Teste standalone — executado, OK
backend/tests/test_alert_rule.py	Teste standalone — executado, OK
backend/tests/test_mqtt_ingestion.py	Teste standalone — executado, OK

Status de verificação

- Ambiente subido via *docker compose up -d timescaledb mosquito* (TimescaleDB + Mosquitto 2, config dev-only em *mosquitto/config/mosquitto.conf* com `allow_anonymous`) e migrations aplicadas com *alembic upgrade head* (0001 + 0002).
- `test_proto_roundtrip.py`: **executado com sucesso** (serialização/desserialização, 19 bytes/leitura, dentro do orçamento LoRaWAN).
- `test_db_connection.py`: **executado com sucesso** contra o TimescaleDB real.
- `test_alert_rule.py`: **executado com sucesso** — abre alerta com janela cheia acima do threshold, fecha ao cair abaixo, e não dispara sem cobertura completa da janela.
- `test_mqtt_ingestion.py`: **executado com sucesso** após corrigir uma condição de corrida no próprio teste (o publisher publicava antes do consumer confirmar o SUBSCRIBE no broker, perdendo as mensagens — sem sessão persistente o Mosquitto não reentrega). Corrigido aguardando o callback `on_subscribe` antes de publicar; nenhuma mudança no consumer de produção. Mensagem malformada foi descartada sem derrubar o consumer, como esperado.
- Suíte reexecutada uma segunda vez em sequência para confirmar reprodutibilidade — 4/4 testes OK nas duas rodadas.

Não incluído nesta fase (próximos passos)

- Notificação externa do alerta (WhatsApp/Email/SMS) — só o registro em ``alerts`` foi implementado.
- Detecção de anomalia (Isolation Forest) — módulo `app/anomaly` ainda vazio.
- Geração de relatório PDF ANP/EPA do produto — módulo `app/reports` ainda vazio.
- API REST e frontend React — ainda não iniciados.

Gerado automaticamente por `backend/scripts/generate_phase2_report.py`